

2024년 1회 정보처리기사 실기 복원 문제 (해설 추가)

출처: <https://chobopark.tistory.com/476> (Life-Journey)

1번

다음 Java 코드에서 알맞는 출력값을 작성하시오.

```
class Connection {
    private static Connection _inst = null;
    private int count = 0;

    static public Connection get() {
        if(_inst == null) {
            _inst = new Connection();
            return _inst;
        }
        return _inst;
    }

    public void count() {
        count++;
    }

    public int getCount() {
        return count;
    }
}

public class main {
    public static void main(String[] args) {
        Connection conn1 = Connection.get();
        conn1.count();
        Connection conn2 = Connection.get();
        conn2.count();
        Connection conn3 = Connection.get();
        conn3.count();
        conn1.count();
        System.out.print(conn1.getCount());
    }
}
```

정답: 4

해설:

이 코드는 **싱글톤(Singleton) 패턴**을 구현한 것입니다. 싱글톤은 "객체를 단 하나만 만들어 모두가 공유"하는 패턴입니다.

핵심 원리:

- static Connection _inst 는 클래스에 하나뿐인 변수입니다.
- get() 은 _inst 가 비어있으면(null) 새로 만들고, 이미 있으면 **기존 객체를 그대로** 돌려줍니다. → 즉 **항상 같은 객체** 하나만 반환됩니다.

따라서 conn1, conn2, conn3는 **이름만 다를 뿐 전부 같은 객체**를 가리킵니다. count() 호출 횟수를 모두 합산합니다.

- conn1.count() → count=1
- conn2.count() → (같은 객체) count=2
- conn3.count() → count=3
- conn1.count() → count=4
- conn1.getCount() → **4**

핵심: 변수 3개가 같은 객체를 공유하므로, count는 누적되어 총 **4번** 증가합니다. "객체가 하나뿐"이라는 싱글톤의 핵심을 이해하면 풀립니다.

2번

다음 C언어 코드에서 알맞는 출력값을 작성하시오.

```
#include <stdio.h>

int main() {
    int v1 = 0, v2 = 35, v3 = 29;
    if(v1 > v2 ? v2 : v1) {
        v2 = v2 << 2;
    } else {
        v3 = v3 << 2;
    }
    printf("%d", v2+v3);
}
```

정답: 151

해설:
삼항 연산자와 비트 시프트 연산을 묻는 문제입니다.

먼저 if 조건의 삼항 연산자 `v1 > v2 ? v2 : v1` :
- `v1 > v2` → `0 > 35` → 거짓 → 콜론 뒤의 값 `v1 (=0)`을 선택합니다.
- if문은 **0이면 거짓, 0이 아니면 참**으로 봅니다. 결과가 0이므로 **else로 갑니다**.

else 실행: `v3 = v3 << 2` ;
- **<< 2 는 왼쪽으로 2비트 시프트**, 즉 **×4**(2의 2제곱)와 같습니다.
- `v3 = 29 × 4 = 116`

마지막: `v2 + v3 = 35 + 116 = 151`

핵심:
- 삼항 연산자 `조건 ? A : B` 는 조건이 참이면 A, 거짓이면 B.
- **<< n 은 ×2ⁿ, >> n 은 ÷2ⁿ**. (시프트는 곱셈/나눗셈의 빠른 버전)

3번

다음은 응집도와 관련해서 보기에서 응집도가 높은 순으로 나열하시오.

보기: ㄱ. 기능 ㄴ. 교환 ㄷ. 우연 ㄹ. 시간

정답: ㄱ, ㄴ, ㄹ, ㄷ

해설:
응집도(Cohesion)는 모듈 내부 요소들이 얼마나 서로 관련되어 한 가지 일에 집중하는지를 나타냅니다. 응집도는 높을수록 좋습니다(좋은 설계).

- 응집도 7단계 (높음 → 낮음):
- 1. 기능적(Functional) — 가장 좋음. 모든 요소가 하나의 목적에 집중.
 - 2. 순차적(Sequential) — 한 요소의 출력이 다음 요소의 입력.
 - 3. 교환적/통신적(Communicational) — 같은 입력/출력 자료를 사용.
 - 4. 절차적(Procedural) — 정해진 순서로 실행.
 - 5. 시간적(Temporal) — 같은 시간대에 실행되는 것끼리.
 - 6. 논리적(Logical) — 비슷한 성격끼리.
 - 7. 우연적(Coincidental) — 가장 나쁨. 아무 관련 없이 모임.

문제의 보기 중에서 높은 순: 기능(1위) > 교환(3위) > 시간(5위) > 우연(7위) → ㄱ, ㄴ, ㄹ, ㄷ

암기 팁(높→낮): "기-순-교(통)-절-시-논-우". 첫 글자만 따서 외우세요. 반대로 결합도는 낮을수록 좋습니다(헛갈리지 말 것).

4번

다음은 C언어에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
#include <stdio.h>
#include <string.h>

void reverse(char* str){
    int len = strlen(str);
    char temp;
    char* p1 = str;
    char* p2 = str + len - 1;
    while(p1 < p2){
        temp = *p1;
        *p1 = *p2;
        *p2 = temp;
        p1++;
        p2--;
    }
}

int main(int argc, char* argv[]){
    char str[100] = "ABCDEFGH";
    reverse(str);
    int len = strlen(str);
    for(int i=1; i<len; i+=2){
        printf("%c", str[i]);
    }
    printf("\n");
    return 0;
}
```

정답: GECA

해설:

두 단계로 나뉩니다: ① 문자열 뒤집기, ② 홀수 인덱스만 출력.

① **reverse 함수**: 양쪽 끝에서 포인터 두 개(p1=앞, p2=뒤)가 서로 마주보며 글자를 바꿔 문자열을 뒤집습니다.
- "ABCDEFGH" → 뒤집으면 **"HGFEDCBA"**

② **출력 for문**: i=1 부터 시작해 i+=2 (2씩 증가), 즉 **홀수 인덱스(1,3,5,7)**만 출력합니다.

뒤집힌 "HGFEDCBA"의 인덱스:

H(0) G(1) F(2) E(3) D(4) C(5) B(6) A(7)

- 인덱스 1 → G
- 인덱스 3 → E
- 인덱스 5 → C
- 인덱스 7 → A

결과: **GECA**

핵심: "두 포인터가 마주보며 교환 = 문자열 뒤집기"의 전형 패턴, 그리고 **i+=2** 는 한 칸씩 건너뛰며 진행한다는 것을 파악하면 됩니다.

5번

아래 그림에서의 네트워크에서 라우터를 통한 할당 가능한 2번, 4번, 5번의 IP를 작성하시오. (※ 원문에 네트워크 이미지 포함)

1) 192.168.35.3/24 / 3) 129.200.10.16/22 / 6) 192.168.36.24/24

보기: 192.168.35.0, 192.168.35.72, 192.168.36.0, 192.168.36.249, 129.200.8.0, 129.200.8.249

정답: 2) 192.168.35.72 / 4) 129.200.8.249 / 5) 192.168.36.249

해설:

서브넷과 호스트 할당 문제입니다. 핵심은 "네트워크 주소와 브로드캐스트 주소는 호스트에 줄 수 없다"는 규칙입니다.

기본 개념:

- IP 주소는 **네트워크 부분 + 호스트 부분**으로 나뉩니다. /24 는 앞 24비트가 네트워크라는 뜻입니다.
- **네트워크 주소**: 호스트 부분이 전부 0 (그 네트워크의 첫 주소). 호스트에 줄 수 없음.
- **브로드캐스트 주소**: 호스트 부분이 전부 1 (마지막 주소). 호스트에 줄 수 없음.
- 그 사이의 주소들만 실제 기기(호스트)에 할당 가능합니다.

보기에서 .0 으로 끝나는 주소(192.168.35.0, 192.168.36.0, 129.200.8.0)는 네트워크 주소라 제외됩니다. 따라서 할당 가능한 일반 호스트 주소는:

- 2) **192.168.35.72** (같은 35.x 대역, /24)
- 4) **129.200.8.249** (3번의 /22 대역에 속하는 호스트)
- 5) **192.168.36.249** (6번의 36.x 대역, /24)

핵심: ".0(네트워크), 맨끝(브로드캐스트)는 호스트에 못 줌"이라는 규칙으로 보기를 거르면 됩니다.

6번

아래 표에서 나타나고 있는 정규형을 작성하시오.

고객아이디	강좌명	강사번호
apple	영어회화	P001
banana	기초토익	P002
carrot	영어회화	P001
carrot	기초토익	P004
orange	영어회화	P003
orange	기초토익	P004

정답: 제3정규형

해설:
정규화(Normalization)는 데이터 중복과 이상현상을 줄이기 위해 테이블을 나누는 과정입니다. 이 표가 어느 단계까지 만족하는지 판단하는 문제입니다.

- 정규형 단계 요약:
- **1NF(제1정규형)**: 모든 값이 원자값(쪼갤 수 없는 하나의 값). 표의 각 칸에 값이 하나씩 → 만족.
 - **2NF(제2정규형)**: 1NF + **부분 함수 종속 제거**(기본 키 일부에만 의존하는 속성 제거).
 - **3NF(제3정규형)**: 2NF + **이행적 함수 종속 제거**(A→B→C 형태의 간접 종속 제거).

이 표는 (고객아이디, 강좌명)이 기본 키이고, 강사번호가 키 전체에 종속됩니다. 이행적 종속도 없으므로 **제3정규형**을 만족합니다.

암기 팁(정규화 순서): "**1NF(원자값) → 2NF(부분종속 제거) → 3NF(이행종속 제거) → BCNF → 4NF(다치종속) → 5NF(조인종속)**". 두문자: "**원-부-이-결-다-조**".

7번

아래의 내용에서 설명하는 네트워크 용어를 영문 약자로 작성하시오.

1. 대표적인 링크 상태 라우팅 프로토콜로, 연결된 링크의 상태를 감시하여 최적의 경로를 선택한다.
2. 단일 자율 시스템 내에서 라우팅 정보를 배포하는 내부 게이트웨이 프로토콜이다.
3. 최단 경로는 Dijkstra 알고리즘을 사용하여 계산된다.

정답: OSPF (Open Shortest Path First)

해설:
OSPF(Open Shortest Path First)는 **링크 상태(Link State)** 방식의 라우팅 프로토콜입니다.

- **링크 상태 방식**: 네트워크 전체의 연결 상태(지도)를 파악한 뒤, **다익스트라(Dijkstra) 알고리즘**으로 최단 경로를 계산합니다.
- **내부 게이트웨이 프로토콜(IGP)**: 하나의 조직(자율 시스템) 내부에서 라우팅에 사용됩니다.
- RIP보다 빠르고 큰 네트워크에 적합합니다.

RIP vs OSPF 비교 (자주 출제):

- **RIP**: 거리 벡터, 홉 카운트, 최대 15홉, 소규모

- OSPF: 링크 상태, 다익스트라, 대규모

키워드 "링크 상태, Dijkstra"는 OSPF입니다.

8번

아래 내용의 각각의 설명에 대한 답을 작성하시오.

- (1) 조인에 참여하는 두 릴레이션의 속성 값을 비교하여 조건을 만족하는 튜플만 반환한다.
- (2) 조건이 정확하게 '=' 등호로 일치하는 결과를 반환한다.
- (3) ((2)) 조인에서 조인에 참여한 속성이 두 번 나오지 않도록 중복된 속성을 제거한 결과를 반환한다.

정답: (1) 세타 조인 / (2) 동등 조인 / (3) 자연 조인

해설:
조인(Join)의 종류를 묻는 문제입니다. 조인은 두 테이블을 연결하는 연산입니다.

- 세타 조인(Theta Join): =, >, <, >= 등 다양한 비교 연산자로 조건을 거는 가장 일반적인 조인입니다. (가장 넓은 개념)
- 동등 조인(Equi Join): 세타 조인 중에서 '=' 등호만 사용하는 조인입니다. 조인 조건이 같은 값일 때만 연결합니다.
- 자연 조인(Natural Join): 동등 조인에서 중복되는 속성(공통 열)을 하나로 합쳐 제거한 결과입니다. 같은 열이 두 번 안 나오게 정리한 것입니다.

관계: 세타 조인 ⊃ 동등 조인 ⊃ 자연 조인 (점점 더 구체적/제한적).

암기 팁: "세타=모든 비교, 동등==만, 자연=중복속성 제거".

9번

다음은 운영체제 페이지 순서를 참고하여 할당된 프레임의 수가 3개일 때 LRU와 LFU 알고리즘의 페이지 부재 횟수를 작성하시오.

페이지 참조 순서: 1, 2, 3, 1, 2, 4, 1, 2, 5, 7

정답: (1) LRU: 6 / (2) LFU: 6

해설:
페이지 교체 알고리즘 문제입니다. 프레임(메모리 칸)이 3개뿐일 때, 새 페이지를 넣으려면 기존 페이지 하나를 내보내야 합니다. 페이지 부재(Page Fault)는 찾는 페이지가 메모리에 없어서 새로 불러오는 횟수입니다.

- LRU(Least Recently Used): 가장 오래 안 쓴(최근에 사용 안 한) 페이지를 내보냅니다.
- LFU(Least Frequently Used): 가장 적게 사용된(사용 횟수가 적은) 페이지를 내보냅니다.

LRU 추적 (프레임 3개, *는 부재):

```
1*: [1]
2*: [1,2]
3*: [1,2,3]
1 : [1,2,3]   (있음, hit) → 1 최근 사용
2 : [1,2,3]   (있음, hit) → 2 최근 사용
4*: 가장 오래 안 쓴 3 제거 → [1,2,4]
1 : hit
2 : hit
5*: 가장 오래 안 쓴 4 제거 → [1,2,5]
7*: 가장 오래 안 쓴 1 제거 → [7,2,5]
```

부재: 1,2,3,4,5,7 → 6회

LFU도 같은 방식으로 사용 빈도가 낮은 것을 내보내며 계산하면 부재 6회가 나옵니다(1,2는 자주 쓰여 남고, 빈도 낮은 것들이 교체됨).

핵심: LRU는 "언제 썼나(시간)", LFU는 "몇 번 썼나(횟수)"가 교체 기준입니다. 표를 그려 한 단계씩 추적하는 것이 정확합니다.

10번

아래 JAVA언어 코드의 실행 순서를 중복 번호 없이 작성하시오.

```

class Parent {
    int x, y;
    Parent(int x, int y) {    // ①
        this.x = x;
        this.y = y;
    }
    int getT() {              // ②
        return x*y;
    }
}

class Child extends Parent {
    int x;
    Child(int x) {            // ③
        super(x+1, x);
        this.x = x;
    }
    int getT(int n){          // ④
        return super.getT()+n;
    }
}

class Main {
    public static void main(String[] args) {    // ⑤
        Parent parent = new Child(3);          // ⑥
        System.out.println(parent.getT());      // ⑦
    }
}

```

실행 순서: 5 → () → () → () → () → ()

정답: 6 3 1 7 2

해설:

프로그램 실행 순서를 추적하는 문제입니다. **상속에서는 부모 생성자가 먼저 실행**된다는 점이 핵심입니다.

흐름:

1. ⑤ main 시작

2. ⑥ new Child(3) 객체 생성 시작

3. ③ Child(int x) 생성자 진입 → 첫 줄 super(x+1, x) 로 **부모 생성자 호출**

4. ① Parent(int x, int y) 생성자 실행 (x, y 설정). 부모 생성자가 끝나야 자식 생성자 나머지가 실행됨

- (Parent 생성 후 Child 생성자의 this.x = x 가 실행되지만 별도 번호 없음)

5. ⑦ parent.getT() 호출

6. ② getT() 실행 → xy 반환

- (주의: Child의 getT(int n) 은 매개변수가 있어 오버로딩*된 별개 메서드입니다. 인자 없는 getT() 를 불렀으므로 부모의 ②가 실행됨)

순서: 5 → 6 → 3 → 1 → 7 → 2

핵심: "객체 생성 시 자식 생성자 → super()로 부모 생성자 먼저 실행 → 부모 끝나고 자식 나머지" 순서. 그리고 getT() (인자 없음)와 getT(int n) (인자 있음)은 다른 메서드라는 오버로딩 함정에 주의하세요.

11번

다음 C언어의 알맞는 출력값을 작성하시오.

```

#include <stdio.h>

typedef struct{
    int accNum;
    double bal;
} BankAcc;

double sim_pow(double base, int year){
    int i;

```

```

    double r = 1.0;
    for(i=0; i<year; i++){
        r = r*base;
    }
    return r;
}

void initAcc(BankAcc *acc, int x, double y){
    acc->accNum = x;
    acc->bal = y;
}

void xxx(BankAcc *acc, double *en){
    if (*en > 0 && *en < acc->bal) {
        acc->bal = acc->bal - *en;
    } else {
        acc->bal = acc->bal + *en;
    }
}

void yyy(BankAcc *acc){
    acc->bal = acc->bal * sim_pow((1+0.1), 3);
}

int main(){
    BankAcc myAcc;
    initAcc(&myAcc, 9981, 2200.0);
    double amount = 100.0;
    xxx(&myAcc, &amount);
    yyy(&myAcc);
    printf("%d and %.2f", myAcc.accNum, myAcc.bal);
    return 0;
}

```

정답: 9981 and 2795.10

해설:

구조체 포인터와 함수 호출 순서를 따라가는 문제입니다.

1. `initAcc(&myAcc, 9981, 2200.0)` → 계좌번호 9981, 잔액 bal=2200.0
2. `xxx(&myAcc, &amount)` (amount=100):
 - 조건 `*en > 0 && *en < acc->bal` → `100 > 0` (참) and `100 < 2200` (참) → 참
 - 따라서 잔액에서 차감: `bal = 2200 - 100 = 2100`
3. `yyy(&myAcc)` :
 - `sim_pow(1.1, 3)` → 1.1을 3번 곱함 = $1.1 \times 1.1 \times 1.1 = 1.331$
 - `bal = 2100 \times 1.331 = 2795.1`
4. 출력: `%d` 로 계좌번호 9981, `%.2f` 로 잔액(소수점 2자리) → **2795.10**

결과: **9981 and 2795.10**

핵심: 구조체 포인터 멤버 접근은 `acc->멤버`. `sim_pow` 는 거듭제곱(base를 year번 곱함) 함수임을 파악하면 됩니다. `%.2f` 는 소수점 둘째 자리까지 표시.

12번

다음 파이썬 코드에 대한 알맞는 출력값을 작성하시오.

```

a = ["Seoul", "Kyeonggi", "Incheon", "Daejun", "Daegu", "Pusan"]
str = "S"

for i in a:
    str = str + i[1]

print(str)

```

정답: Seynaau

해설:

리스트의 각 문자열에서 **두 번째 글자(인덱스 1)**를 뽑아 이어 붙이는 문제입니다.

- `str` 은 처음에 "S"로 시작합니다.
- `for`문은 리스트 `a`의 각 단어를 `i` 에 담고, `i[1]` (그 단어의 **인덱스 1 = 두 번째 글자**)을 `str`에 붙입니다.

각 단어의 두 번째 글자:

- "Seoul" → **S**eoul → 'e'
- "Kyeonggi" → **K**yeonggi → 'y'
- "Incheon" → **I**ncheon → 'n'
- "Daejun" → **D**aejun → 'a'
- "Daegu" → **D**aegu → 'a'
- "Pusan" → **P**usan → 'u'

이어 붙이면: "S" + "e" + "y" + "n" + "a" + "a" + "u" = "**Seynaau**"

핵심: 인덱스는 0부터 시작하므로 `i[1]` 은 **두 번째** 글자입니다. 누적 변수(`str`)에 한 글자씩 더해가는 패턴입니다.

13번

아래 보기의 SQL 문장과 테이블을 참고하여 출력값을 표로 작성하시오. (※ 원문에 테이블 이미지 포함)

```
SELECT B
FROM R1
WHERE C IN (SELECT C FROM R2 WHERE D="k");
```

정답:

B

a
b

해설:

서브쿼리(중첩 질의)를 사용한 SQL입니다. 안쪽 쿼리부터 풀고 바깥쪽으로 나옵니다.

작동 순서:

1. **안쪽 쿼리** `SELECT C FROM R2 WHERE D="k"` → R2 테이블에서 D가 "k"인 행들의 C 값을 먼저 구합니다.
2. **바깥쪽 쿼리** `SELECT B FROM R1 WHERE C IN (...)` → R1 테이블에서, C 값이 위에서 구한 C 목록 안에 있는(IN) 행들의 B 값을 출력합니다.

- **IN 연산자**: "괄호 안 목록 중 하나라도 일치하면 참"이라는 뜻입니다.

테이블에 따라 조건을 만족하는 B 값이 'a', 'b'이므로 결과는 a, b 두 행입니다.

핵심: 서브쿼리는 **안에서 밖으로** 읽습니다. "안쪽 결과 목록 → IN으로 바깥쪽에서 걸러내기" 흐름을 이해하세요.

14번

아래는 애플리케이션 테스트 관리에 대한 내용이다. 설명하는 답을 보기에서 골라 작성하시오.

1. 모든 분기와 조건의 조합을 고려하나, 테스트가 필요한 중요한 조합을 찾아내는 데 중점을 둔다.
2. 특정 조건을 수행할 때 다른 조건과 상관없이 전체 결과에 영향을 미치는 조건만 테스트한다.
3. 각각의 파라미터는 적어도 한 번은 최종 결과에 영향을 주어야 한다.

보기: ㄱ. 구문 커버리지 ㄴ. 결정 커버리지 ㄷ. 조건 커버리지 ㄹ. 변경 조건/결정 커버리지 ㄴ. 다중 조건 커버리지 ㄷ. 경로 커버리지 ㄴ. 조건/결정 커버리지

정답: ㄹ. 변경 조건/결정 커버리지

해설:

변경 조건/결정 커버리지(MC/DC, Modified Condition/Decision Coverage)를 묻는 문제입니다.

- 지문의 세 설명 모두 MC/DC의 특징입니다.

- **MC/DC:** 각 **개별 조건이 독립적으로 전체 결과에 영향**을 미치는지 확인하는 커버리지입니다. "다른 조건과 상관없이 그 조건 하나가 결과를 바꿀 수 있는가"를 테스트합니다.
- 조건/결정 커버리지보다 강력하고, 다중 조건 커버리지보다는 테스트 수가 적어 **효율적**입니다. 항공·국방 등 고신뢰 소프트웨어에서 요구됩니다.

커버리지 강도 순서(약→강):
문장 < 결정(분기) < 조건 < 조건/결정 < 변경조건/결정(MC/DC) < 다중조건

키워드 "**각 조건이 독립적으로 결과에 영향**"은 **MC/DC**입니다.

15번

다음 아래 내용을 보고 보기에서 알맞는 용어를 골라 작성하시오.

인터넷 공격자의 존재를 숨기면서 시스템에 대한 무제한 접근 권한을 부여하는 악성 프로그램. 펌웨어·가상화 계층 등에서 작동하며 운영체제의 시스템콜을 해킹하여 악성코드의 실행 여부를 숨겨 안티바이러스 탐지를 우회한다.

보기: ㄱ. Worm ㄴ. Trojan horse ㄷ. Backdoor ㄹ. Virus ㅁ. Ransomware ㅂ. Spyware ㅅ. Rootkit

정답: ㅅ (Rootkit)

해설:
루트킷(Rootkit)은 자신의 존재를 **숨기면서** 시스템에 대한 **최고 권한(root)**을 몰래 차지하는 악성 프로그램입니다.

- "Root(최고 관리자 권한) + Kit(도구 모음)"의 합성어입니다.
- 운영체제 깊숙한 곳(커널, 시스템 콜)에 숨어, 자신의 파일·프로세스를 안 보이게 만들어 **백신 탐지를 회피**합니다.
- 한 번 심어지면 공격자가 언제든지 들어올 수 있는 무제한 접근 권한을 제공합니다.

다른 악성코드와 구분:
- **랜섬웨어:** 파일을 암호화하고 돈을 요구
- **스파이웨어:** 몰래 정보를 빼냄
- **백도어:** 인증 없이 들어오는 뒷문
- **루트킷:** 탐지를 회피하며 **최고 권한을 숨김** ← 키워드 "탐지 우회, 은폐, 무제한 권한"

16번

다음 Java 코드를 보고 알맞는 출력값을 작성하시오.

```
class classOne {
    int a, b;
    public classOne(int a, int b) {
        this.a = a;
        this.b = b;
    }
    public void print() {
        System.out.println(a + b);
    }
}

class classTwo extends classOne {
    int po = 3;
    public classTwo(int i) {
        super(i, i+1);
    }
    public void print() {
        System.out.println(po*po);
    }
}

public class main {
    public static void main(String[] args) {
        classOne one = new classTwo(10);
        one.print();
    }
}
```

```
}

```

정답: 9

해설:
오버라이딩의 핵심을 묻는 문제입니다.

- classOne one = new classTwo(10); → 변수 타입은 classOne이지만, 실제 객체는 classTwo입니다.
- one.print() 를 부르면, 오버라이딩된 메서드는 실제 객체(classTwo) 기준으로 실행됩니다. → classTwo의 print()가 실행됩니다.
- classTwo의 print()는 po * po 를 출력합니다. po=3이므로 3 × 3 = 9

(생성자에서 super(i, i+1) 로 a=10, b=11이 설정되지만, classTwo의 print()는 a, b가 아니라 po를 쓰므로 출력에 영향 없음)

핵심: "부모 타입 변수 + 자식 객체 → 자식의 오버라이딩 메서드 실행". 이를 동적 바인딩(다형성)이라 부르며 자바 객체지향의 가장 중요한 개념입니다.

17번

다음 아래 내용을 보고 보기에서 알맞는 용어를 골라 작성하시오.

- 불특정 다수가 아닌 명확한 표적을 정하여 지속적인 정보수집 후 공격을 감행한다.
- 시스템 직접 침투뿐 아니라 표적 내부직원들이 이용하는 다양한 단말을 대상으로 한다.
- Zero-day 취약점, 악성코드 등 다양한 보안 위협 공격 기술을 사용한다.
- 일반적으로 침투, 검색, 수집 및 유출의 4단계로 실행된다.

보기: ㄱ. 사회공학 기법 ㄴ. Adware ㄷ. MITM ㄹ. XDR ㅁ. Replace attack ㅂ. key logger attack ㅅ. APT

정답: ㅅ (APT)

해설:
APT(Advanced Persistent Threat, 지능형 지속 위협) 공격을 묻는 문제입니다.

- Advanced(지능형): Zero-day 취약점 등 고도의 기술 사용
- Persistent(지속적): 한 번에 끝내지 않고 장기간 잠복하며 정보를 수집
- Threat(위협): 명확한 특정 표적을 정해 끈질기게 공격

특징:
- 불특정 다수가 아닌 특정 조직/기업을 노립니다.
- 시스템뿐 아니라 내부 직원(이메일, 단말기)을 통해 침투합니다.
- 보통 침투 → 검색 → 수집 → 유출의 단계로 진행됩니다.

키워드 "특정 표적, 지속적, Zero-day"는 APT입니다.

18번

아래의 SQL 코드와 테이블을 참고하여 결과값을 작성하시오. (※ 원문에 테이블 이미지 포함)

```
SELECT COUNT(*)
FROM TABLE
WHERE EMPNO > 100 AND SAL >= 3000 OR EMPNO = 200

```

정답: 1

해설:
WHERE 조건에서 AND와 OR의 우선순위를 묻는 문제입니다.

핵심 규칙: AND가 OR보다 먼저 계산됩니다(AND 우선순위가 높음). 따라서 조건은 다음처럼 묶입니다:

```
(EMPNO > 100 AND SAL >= 3000) OR (EMPNO = 200)

```

즉, "사번이 100 초과이면서 급여가 3000 이상"이거나, "사번이 정확히 200"인 행을 셉니다. 두 조건 중 하나라도 만족하면 카운트됩니다.

테이블에서 이 조건을 만족하는 행이 1개이므로 COUNT(*) 는 1.

핵심 암기: "**AND 먼저, OR 나중**". 헷갈리면 AND를 곱셈(×), OR을 덧셈(+)처럼 생각하세요. 명확히 하려면 괄호를 직접 치는 습관이 좋습니다.

19번

다음 C언어 코드의 알맞는 출력값을 작성하시오.

```
#include <stdio.h>
#include <ctype.h>

int main(){
    char* p = "It is 8";
    char result[100];
    int i;

    for(i=0; p[i]!='\0'; i++){
        if(isupper(p[i]))
            result[i] = (p[i]-'A'+5) % 25 + 'A';
        else if(islower(p[i]))
            result[i] = (p[i]-'a'+10) % 26 + 'a';
        else if(isdigit(p[i]))
            result[i] = (p[i]-'0'+3) % 10 + '0';
        else if(!(isupper(p[i]) || islower(p[i]) || isdigit(p[i])))
            result[i] = p[i];
    }

    result[i] = '\0';
    printf("%s\n", result);
    return 0;
}
```

정답: Nd sc 1

해설:

글자 종류에 따라 다른 **암호화(자리 이동)**를 적용하는 문제입니다. 각 함수의 의미:

- `isupper`: 대문자인가? / `islower`: 소문자인가? / `isdigit`: 숫자인가?

변환 규칙 (각 글자를 0부터 시작하는 번호로 바꿔 계산 후 다시 문자로):

- 대문자: (글자- 'A'+5) % 25 + 'A' → 알파벳 순서에서 +5 이동
- 소문자: (글자- 'a'+10) % 26 + 'a' → +10 이동
- 숫자: (글자- '0'+3) % 10 + '0' → +3 이동
- 그 외(공백 등): 그대로

"It is 8"을 한 글자씩:

- 'I'(대문자): I는 A부터 8번째(0기준 8). (8+5)%25=13 → A+13 = '**N**'
- 't'(소문자): t는 a부터 19번째. (19+10)%26=3 → a+3 = '**d**'
- ' '(공백): 그대로 ' '
- 'i'(소문자): i는 8번째. (8+10)%26=18 → a+18 = '**s**'
- 's'(소문자): s는 18번째. (18+10)%26=2 → a+2 = '**c**'
- ' '(공백): 그대로 ' '
- '8'(숫자): (8+3)%10=1 → '0'+1 = '**1**'

결과: "**Nd sc 1**"

핵심: 문자 - 'A' 는 그 글자가 알파벳 몇 번째인지(0부터)를 구하는 방법입니다. % 개수 로 알파벳을 한 바퀴 돌리고, 다시 + 'A' 로 문자로 되돌립니다. 이것이 시저 암호(자리 이동 암호)의 원리입니다.

20번

다음 아래의 내용을 보고 알맞는 용어를 작성하시오.

구체적인 클래스에 의존하지 않고 서로 연관·의존적인 객체들의 조합을 만드는 인터페이스를 제공하는 패턴. 연관성 있는 객체 군을 추상화하고 구체적 상황이 주어지면 팩토리 객체에서 묶은 객체 군을 구현화하는 생성 패턴이다. kit라고도 불린다.

정답: Abstract Factory

해설:

추상 팩토리(**Abstract Factory**)는 GoF 생성 패턴 중 하나입니다.

- 서로 관련 있는 객체들의 "묶음(군)"을 통째로 생성하는 인터페이스를 제공합니다.
- 구체적인 클래스 이름에 의존하지 않고, 상황에 맞는 객체 군을 만들어냅니다.
- 비유: "겨울 테마 가구 세트(의자+책상+조명)"를 한꺼번에 만드는 공장과, "여름 테마 세트"를 만드는 공장을 따로 두는 것. 어떤 공장을 쓰느냐에 따라 일관된 세트가 생성됩니다.
- 별칭으로 "**Kit**"라고도 불립니다.

비교: 팩토리 메서드(**Factory Method**)는 객체 하나의 생성을 서브클래스에 위임하는 패턴이고, 추상 팩토리는 관련된 여러 객체 군을 만드는 더 큰 개념입니다.

키워드 "연관된 객체 군, kit"는 **Abstract Factory**입니다.